

AGENTIC AI CHAT APPLICATION

Complete From-Scratch to Current-State Documentation

Developer: Rohit Rajput

GitHub: <https://github.com/Sarvanrohit05>

Backend: Spring Boot 4.1.1 + Java 17 + Spring AI 2.0.1

Frontend: Angular 21

Database: MySQL + Spring Data JPA/Hibernate

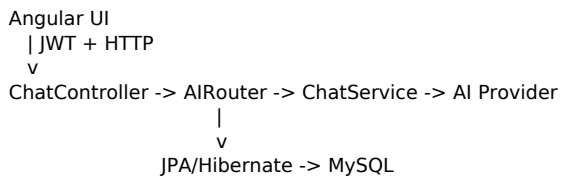
AI: Groq, NVIDIA NIM, Gemini, Ollama

Ye document project ko setup se final UI/file-processing state tak Hinglish me explain karta hai. Har major class/method ka purpose, data flow, database behavior, testing issues aur interview explanation included hai.

1. Project ka Goal

Target sirf chatbot banana nahi tha. Aim ek full-stack AI application banana tha jisme secure login, multiple AI providers, automatic routing/fallback, persistent chat history, tool calling, file processing aur ChatGPT-style Angular UI ho.

High-level flow:



GitHub: <https://github.com/Sarvanrohit05>

2. Technology Stack

Layer	Technology	Purpose
Language	Java 17	Backend runtime
Backend	Spring Boot 4.1.1	REST, DI, application
AI	Spring AI 2.0.1	ChatClient abstraction
Providers	Groq / NVIDIA / Gemini / Ollama	Routing + fallback + local option
Frontend	Angular 21	SPA chat interface
DB	MySQL	Persistence
ORM	Spring Data JPA / Hibernate	Entity/repository mapping
Security	Spring Security + JWT	Stateless authentication
PDF	PDFBox 3.0.5	PDF text extraction
Build	Maven	Dependencies/build

3. Project Setup and Environment

Backend package root: com.in.rohit.spring_ai_agent. Maven project me Spring Web, Spring AI, JPA, MySQL, Security, JWT aur provider dependencies add ki gayi.

API keys source code me hard-code nahi ki gayi. Environment variables: GEMINI_API_KEY, GROQ_API_KEY, NVIDIA_API_KEY.

```
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.options.model=llama3.2
spring.servlet.multipart.max-file-size=20MB
spring.servlet.multipart.max-request-size=20MB
```

Ollama llama3.2 CPU-only par slow tha, isliye optional/local provider rakha gaya. Architecture external Ollama server ko base URL change karke support kar sakta hai.

4. AI Provider Integration

Provider	Current role	Important point
Gemini	Original primary; later final fallback	Spring AI Google model
Groq	Tool/simple/general requests + fallback	OpenAI-compatible
NVIDIA NIM	Long/complex requests + fallback	base URL ends /v1
Ollama	Optional local model	llama3.2 on localhost

Development timeline: initial fixed fallback Gemini → Groq → NVIDIA tha. Later intelligent routing add hui: tool/simple/greeting/short requests Groq, long/complex requests NVIDIA, provider failure par Groq → NVIDIA → Gemini fallback.

NVIDIA ka working base URL: <https://integrate.api.nvidia.com/v1>. Direct API test me HTTP 200 verify hua.

5. AiConfig — Provider Beans

File: AiConfig.java Class: AiConfig

Same ChatClient type ke multiple beans hone ki wajah se named beans aur @Qualifier use kiye gaye. Isse provider configuration service/router se separate rehti hai.

```
@Bean("groqChatClient")
public ChatClient groqChatClient(@Qualifier("openAiChatModel") ChatModel model) {
    return ChatClient.builder(model).build();
}

@Bean("nvidiaChatClient")
public ChatClient nvidiaChatClient() {
    OpenAiChatModel model = OpenAiChatModel.builder()
        .options(OpenAiChatOptions.builder()
            .baseUrl("https://integrate.api.nvidia.com/v1")
            .apiKey(System.getenv("NVIDIA_API_KEY"))
            .model("openai/gpt-oss-20b")
            .maxTokens(1000).build())
        .build();
    return ChatClient.builder(model).build();
}
```

WHY: ChatService ko provider setup details nahi pata hone chahiye; usko named ChatClient milna chahiye.

6. MyTools — Agentic Tool Calling

File: MyTools.java **Class:** MyTools

@Tool methods currentTime() aur add(double,double) banaye gaye. Model deterministic operation ke liye backend function invoke kar sakta hai.

```
@Component
public class MyTools {
    @Tool
    public String currentTime() { /* current date/time */ }

    @Tool
    public double add(double a, double b) {
        return a + b;
    }
}
```

Gemini aur Groq calls me .tools(tools) pass hota hai. Interview me isko tool/function calling ya backend function orchestration bol sakte ho.

7. DTO + Controller + REST API

Files: dto/ChatRequest.java, dto/ChatResponse.java, ChatController.java

```
POST /api/chat
{
  "conversationId": "abc123",
  "message": "What is Java?"
}
```

ChatController thin rakha gaya: request receive → AIRouter.route() → ChatResponse return.

```
@PostMapping
public ChatResponse chat(@RequestBody ChatRequest request) {
    String response = aiRouter.route(request.getConversationId(), request.getMessage());
    return new ChatResponse(request.getConversationId(), response);
}
```

Later controller me /history, /history/{conversationId}, DELETE aur /upload endpoints add hue.

8. ChatService — Core Business Layer

File: ChatService.java Class: ChatService

Method	Why it exists
saveUserMessage	Conversation create/ownership check + USER row save
saveAiMessage	Successful AI response ko AI role ke saath save
getChatHistory	Messages ko chronological text context me convert
getMessages	Conversation messages + ownership validation
getUserConversationsByEmail	Logged-in user ki chats
getChatHistoryForUser	User-specific history response
chatWithGemini	Gemini provider call
chatWithGroq	Groq + tools provider call
chatWithNvidia	NVIDIA provider call
chatWithGeminiVision	Image bytes + MIME type to Gemini vision
deleteChat	Messages delete + Conversation delete; @Transactional

Critical design: USER message router me ek baar save hota hai. Provider-specific methods me save nahi hota; warna fallback par duplicate USER records ban sakte the.

9. AIRouter — Routing and Fallback

File: AIRouter.java **Class:** AIRouter

Detection	Route
time / calculate / add / sum	GROQ
Greeting	GROQ
<=150 characters	GROQ
Long/complex	NVIDIA
Selected provider failure	GROQ → NVIDIA → Gemini

route() ka logical sequence: 1) old history load, 2) current USER save, 3) prompt build, 4) route detect, 5) selected provider call, 6) failure par fallback, 7) success par AI save, 8) response return.

AIRouter orchestration owner hai; ChatService provider calls/persistence owner hai. Ye separation interview me important hai.

10. Database Design

Three main tables: users, conversations, messages.

Table	Main fields	Meaning
users	id, username, email, password, role, active, created_at, updated_at	Application entity
conversations	id, conversation_id, user_id, title, created_at	One row per chat
messages	id, conversation_id, content, role, created_at	USER/AI history

Relationship: User 1 → many Conversations → many Messages. Conversation.id DB primary key hai; conversationId unique business/API identifier hai.

Current Conversation entity @ManyToOne(fetch = LAZY) User ke saath mapped hai.

@JoinColumn(name="user_id", nullable=false) use hota hai. DB-level FK intentionally add nahi kiya gaya; ownership application/JPA level par validate hoti hai.

11. Repositories

ConversationRepository: conversation lookup, exists check, user-wise list, ownership lookup aur delete.

```
Optional<Conversation> findByConversationId(String conversationId);
boolean existsByConversationId(String conversationId);
List<Conversation> findByUserEmail(String email);
Optional<Conversation> findByConversationIdAndUserEmail(String conversationId, String email);
void deleteByConversationId(String conversationId);
```

MessageRepository: chronological history + delete.

```
List<Message> findByConversationIdOrderByCreatedAtAsc(String conversationId);
@Modifying
void deleteByConversationId(String conversationId);
```

Spring Data JPA derived query names automatically query implementation generate karte hain.

12. Chat History Flow

First request: conversationId=abc123

USER: My name is Rohit

DB:

Conversation abc123

USER message saved

AI response saved

Second request:

USER: What is my name?

History loaded:

USER: My name is Rohit

AI: Nice to meet you Rohit

New prompt = old history + current USER

Same conversationId se new Conversation row nahi banti; additional Message rows banti hain. New conversationId se new Conversation row banti hai.

History database se fetch hoti hai using findByConversationIdOrderByCreatedAtAsc, then role + content ko newline separated context me convert kiya jata hai.

13. JWT Authentication and Ownership

Files: SecurityConfig.java, JwtService.java, JwtAuthenticationFilter.java, auth classes, Angular auth.interceptor.ts

Login ke baad JWT generate hota hai; subject email hai; expiration 1 hour. Angular protected requests me Authorization: Bearer token bhejta hai.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .anyRequest().authenticated()
)
```

JwtAuthenticationFilter token parse karke email nikalta hai aur SecurityContext me Authentication set karta hai. Controller/service authenticated email se conversation ownership validate karte hain.

Production note: current static JWT secret ko environment/secret manager me move karna chahiye.

14. Conversation Title and User-wise History

Conversation entity me title add hua. New conversation create hote waqt first user message ko title banaya gaya, taaki sidebar me meaningful text dikhe.

```
public Conversation(String conversationId, User user, String title) {  
    this.conversationId = conversationId;  
    this.user = user;  
    this.title = title;  
    this.createdAt = LocalDateTime.now();  
}
```

ConversationResponse me conversationId, title, username, email aur createdAt fields hain. Angular GET /api/chat/history se current user ki chats load karta hai.

15. Delete Chat — Why @Transactional

Delete me pehle messages aur phir conversation delete hota hai. Ownership pehle authenticated email se validate hoti hai.

```
@Transactional
public void deleteChat(String conversationId, String email) {
    Conversation conversation = conversationRepository
        .findByConversationIdAndUserEmail(conversationId, email)
        .orElseThrow(...);

    messageRepository.deleteByConversationId(conversationId);
    conversationRepository.delete(conversation);
}
```

@Transactional logical delete sequence ko ek transaction me rakhta hai.

MessageRepository.deleteByConversationId par @Modifying add kiya gaya. User ne final state me confirm kiya ki delete working hai.

16. Angular Frontend

Frontend folder: G:\Agentic-AI-ProjectFrtotends\angular-ai-agent. Angular 21 standalone app.

File	Responsibility
app.config.ts	HttpClient + interceptor
app.routes.ts	Login/Chat routes
auth.interceptor.ts	Bearer JWT header
chat.service.ts	Chat/history/upload/delete APIs
chat.ts	UI state/actions
chat.html	UI structure
chat.css	Responsive styling

Main route / login component aur / chat component ko use karta hai; wildcard login/chat routing setup ke according redirect hota hai.

17. Angular ChatService and Interceptor

```
sendMessage(conversationId, message)
getHistory(conversationId)
getConversations()
deleteChat(conversationId)
uploadFile(file, conversationId, message)
```

uploadFile FormData use karta hai: file + conversationId + message. responseType text use hua kyunki upload endpoint text response return karta hai.

```
const token = localStorage.getItem('token');
if (token && !req.url.includes('/api/auth/')) {
  req = req.clone({
    headers: { Authorization: `Bearer ${token}` }
  });
}
```

Browser Network tab me Authorization header verify kiya gaya.

18. Final Chat UI and File Preview

Sidebar: AI Agent, New Chat, conversation list, delete icon, Developed by Sarvanrohit05, GitHub link aur Logout.

Main area: AI Assistant header, messages, loading indicator, attachment button, text input aur Send.

Important UI fix: selected file pehle input-area me absolute position se show ho raha tha aur layout overlap kar raha tha. Final implementation me selectedFile ko messages/blank chat area ke andar .file-preview card ke roop me move kiya gaya.

```
@if (selectedFile) {  
  <div class="file-preview">  
    <div class="file-icon">📎</div>  
    <div class="file-info">  
      <div class="file-name">{{ selectedFile.name }}</div>  
      <div class="file-status">File attached</div>  
    </div>  
  </div>  
}
```

GitHub branding: <https://github.com/Sarvanrohit05>

19. Image Analysis Flow

Upload endpoint contentType detect karta hai. image/* hone par bytes read hote hain aur Gemini vision request me media ke roop me pass hote hain.

```
MimeType mimeType = MimeTypeUtils.parseMimeType(contentType);
ByteArrayResource imageResource = new ByteArrayResource(imageBytes);
return geminiChatClient.prompt()
    .user(user -> user.text(message).media(mimeType, imageResource))
    .call().content();
```

Image binary ko current design me MySQL message table me directly store nahi kiya gaya. Chat history me textual marker jaise [Image: filename] save kiya ja sakta hai. Image analysis aur PDF extraction separate processing paths hain.

20. PDF Extraction Flow

File: service/PdfTextExtractorService.java Class: PdfTextExtractorService

```
@Service
public class PdfTextExtractorService {
    public String extractText(MultipartFile file) throws IOException {
        byte[] fileBytes = file.getBytes();
        try (PDDocument document = Loader.loadPDF(fileBytes)) {
            PDFTextStripper stripper = new PDFTextStripper();
            return stripper.getText(document);
        }
    }
}
```

Dependency: org.apache.pdfbox:pdfbox:3.0.5. application/pdf branch extractor call karta hai. Blank text ho to readable text nahi mila response; otherwise extracted text return.

Debugging issue: runtime ClassNotFoundException: org.apache.pdfbox.Loader at PdfTextExtractorService.java line 24. Ye PDF processing logic se zyada runtime classpath/dependency issue tha. Maven Update/Force Update/Clean/restart aur mvn dependency:tree se resolved PDFBox version verify karna correct debugging path hai.

20MB upload limits application.properties me set kiye gaye. Pehle upload error MaxUploadSizeExceededException tha, JWT/403 issue nahi.

21. Testing, Errors and Fixes

Issue	Cause	Resolution
Ollama slow	CPU-only llama3.2	Optional/local provider; cloud providers continue
NVIDIA API issue	Base URL/path	/v1 base URL verified
Upload size error	Multipart limit	20MB limits
PDF Loader missing	Runtime dependency/classpath	Maven refresh + dependency tree
Delete failed	Transaction/delete sequence	@Transactional + @Modifying
Unauthorized chat access	Ownership missing	findByConversationIdAndUserEmail
Duplicate USER risk	Save inside provider methods	Save once in router
Conversation duplicates	No existence check	existsByConversationId
File chip overlap	Wrong DOM position	Move to messages + .file-preview

Testing tools: Postman for REST, MySQL queries for persistence, browser Network tab for JWT/upload/delete, and Angular UI for end-to-end behavior.

22. Current Project Structure

Backend

com.in.rohit.spring_ai_agent

- ├── dto
 - ├── ChatRequest.java
 - └── ChatResponse.java
- ├── entity
 - ├── User.java
 - ├── Conversation.java
 - └── Message.java
- ├── repository
 - ├── UserRepository.java
 - ├── ConversationRepository.java
 - └── MessageRepository.java
- ├── service
 - └── PdfTextExtractorService.java
- ├── MyTools.java
- ├── ChatController.java
- ├── ChatService.java
- ├── AIRouter.java
- ├── AiConfig.java
- ├── SecurityConfig.java
- ├── JwtService.java
- └── JwtAuthenticationFilter.java

Frontend

angular-ai-agent/src/app

- ├── auth.interceptor.ts
- ├── chat.service.ts
- ├── app.config.ts
- ├── app.routes.ts
- ├── chat/chat.ts
- ├── chat/chat.html
- ├── chat/chat.css
- └── login/...

Auth package ke exact local filenames workspace ke final arrangement par depend kar sakte hain; responsibilities upar documented hain.

23. Complete End-to-End Flow

- 1 Login → JWT
- 2 Angular stores token
- 3 User sends message
- 4 Interceptor adds Bearer token
- 5 POST /api/chat
- 6 ChatController
- 7 ARouter loads history
- 8 USER message saved
- 9 detectRoute()
- 10 Groq/NVIDIA selected
- 11 ChatService provider call
- 12 Failure → fallback
- 13 Success → AI message saved
- 14 Response → Angular
- 15 UI renders
- 16 MySQL retains history

New conversation me authenticated User ke saath Conversation create hoti hai. Existing conversation me same Conversation reuse hoti hai. Upload me content type ke hisaab se image/PDF branch select hoti hai. Delete me ownership → messages delete → conversation delete.

24. Interview Explanation — Hinglish

30-second answer: “Maine Spring Boot aur Angular based Agentic AI chat application develop kiya hai. Spring AI ke through Groq, NVIDIA NIM, Gemini aur optional Ollama integrate kiye. Application request ki complexity/type ke basis par provider route karti hai aur failure par fallback karti hai. JWT authentication se user-specific conversations secure hain. Spring Data JPA/MySQL me conversations aur USER/AI messages persist hote hain. Spring AI tool calling se current time aur calculation jaise backend functions invoke hote hain. Angular UI me history, delete chat, file upload aur image/PDF processing implemented hai.”

Why AIRouter? Provider selection/fallback/orchestration ko controller me mix nahi karna tha. AIRouter decision layer hai.

Why ChatService? AI calls + persistence + chat-related business operations ko centralize karna aur controller ko thin rakhna.

How memory? Messages MySQL me save hote hain; same conversationId ke old messages chronological order me fetch karke current prompt ke saath provider ko bheje jate hain.

How image? MultipartFile → bytes + MIME → Gemini media input → response → AI message persistence.

How PDF? MultipartFile → PDFBox Loader → PDDocument → PDFTextStripper → extracted text.

25. Resume-Ready Description

Agentic AI Chat Application | Spring Boot, Angular, Spring AI, JWT, MySQL

- Developed a full-stack AI conversational application using Spring Boot, Angular and Spring AI.
- Integrated Groq, NVIDIA NIM, Gemini and optional Ollama with intelligent provider routing and fallback handling.
- Implemented Spring AI tool/function calling for deterministic backend operations such as time and calculation.
- Implemented JWT-based authentication and user-specific conversation ownership.
- Persisted users, conversations and chronological USER/AI messages using Spring Data JPA, Hibernate and MySQL.
- Built chat history, conversation title, delete chat and user-wise conversation APIs.
- Implemented multipart file upload with Gemini image analysis and PDF text extraction using PDFBox.
- Built responsive ChatGPT-style Angular UI with attachment support and GitHub developer branding.

GitHub: <https://github.com/Sarvanrohit05>

26. Future Production Roadmap

- Streaming AI responses using SSE/WebSocket.
- Centralized @RestControllerAdvice error handling.
- Move JWT/API secrets to environment or secret manager.
- Dockerize backend, frontend and MySQL.
- Deploy backend/frontend; run Ollama on a separate GPU/server if required.
- Add Redis caching/rate limiting.
- Add JUnit and integration tests.
- Add RAG: embeddings + vector store + document chunking for large PDFs.
- Add object storage for uploaded files if persistent file history is needed.
- Add provider health checks, metrics and CI/CD.

Current project already covers Full Stack + AI integration + tool calling + routing + fallback + security + database + file processing. Production-level next step: tests, deployment, streaming and RAG.

Appendix — Exact Files to Remember

File/Class	Interview responsibility
AiConfig	Named ChatClient/provider configuration
AIRouter	Routing, orchestration and fallback
ChatService	AI calls, persistence, ownership, vision
ChatController	REST + upload entry point
MyTools	@Tool functions
Conversation	Chat metadata + User relation
Message	USER/AI history rows
User	Authentication identity
ConversationRepository	Conversation queries/ownership/delete
MessageRepository	Ordered history/delete
PdfTextExtractorService	PDFBox text extraction
SecurityConfig	JWT stateless security rules
JwtService	JWT create/parse
JwtAuthenticationFilter	Bearer token to Authentication
chat.service.ts	Angular API client
auth.interceptor.ts	Bearer header
chat.ts	UI state/actions
chat.html	UI
chat.css	Responsive styling

Final GitHub reference: <https://github.com/Sarvanrohit05>

Source note: Document prior project documentation aur current implementation context ko combine karta hai. Jahan architecture evolve hua, earlier aur current behavior ko separately explain kiya gaya hai; unsupported details ko intentionally avoid kiya gaya hai.